

LAB 5

Load Balanced Back-end

Reverse Proxy, Health-Aware Round Robin, WebSocket Integration, and Performance Experiments

Student Name	Sneha Nagmoti
Roll Number	12342090
Assignment	Lab 5: Load Balanced Back-end
Implementation	Go load balancer and load generator; FastAPI messaging replicas
Report Date	24 August 2026

VERIFIED IMPLEMENTATION

This report is generated from the repository's current source and final remote experiment artifacts. Verification used the assigned Sys1-Sys4 machines, three live HTTPS backends, health failover and recovery, and a real WebSocket connection through Sys1.

1. Executive Summary

The assignment has been implemented as a complete load-balanced messaging back-end. Sys1 runs a Go reverse proxy. Sys2, Sys3, and Sys4 run replicas of the previous FastAPI messaging server. A concurrent Go load generator executes fair one-backend and three-backend experiments and writes JSON plus CSV results.

The audited implementation uses validated backend URLs, atomic health and in-flight state, health-aware round robin, active health checks, total request timeouts, explicit TLS controls, WebSocket-safe response handling, browser affinity for stateful chat sessions, and bounded latency percentile metrics.

Assignment traceability

Assignment requirement	Implementation	Verification
Load balancer on Sys1	Go command at cmd/load-balancer; listens on container port 4000, mapped publicly to 4237.	Remote process plus public health, status, metrics, and routing checks.
Backends on Sys2, Sys3, Sys4	FastAPI messaging server deployed on HTTPS port 5000.	All three assigned machines passed remote health checks.
Local load generator	Concurrent Go worker pool targets 10.1.75.53:4237 from the student workstation.	Mapped-port TLS run plus JSON/CSV writers.
Measure Sys2 only	run_mapped_experiments.py starts Sys1 with only backend 1 and drives it from the local PC.	Recorded 1_backend.json and CSV row.
Measure all three	The same mapped-port script starts Sys1 with all three backend URLs.	Recorded 3_backends.json and CSV row.
Comparison table	Section 6 uses the recorded artifacts and calculated improvements.	Numbers cross-checked against JSON.
Messaging app integration	REST, uploads, and WebSocket upgrades route through the LB with affinity.	Remote register, room, token rotation, and reconnecting WebSocket acceptance test.
Single PDF report	This generated PDF contains systems, code, results, analysis, and evidence.	Rendered and visually inspected before submission.

2. System Architecture

The local load generator sends requests to Sys1. Sys1 selects a healthy backend using round robin and proxies the request or upgraded WebSocket connection. Health checks remove failed replicas from scheduling and restore them after recovery.

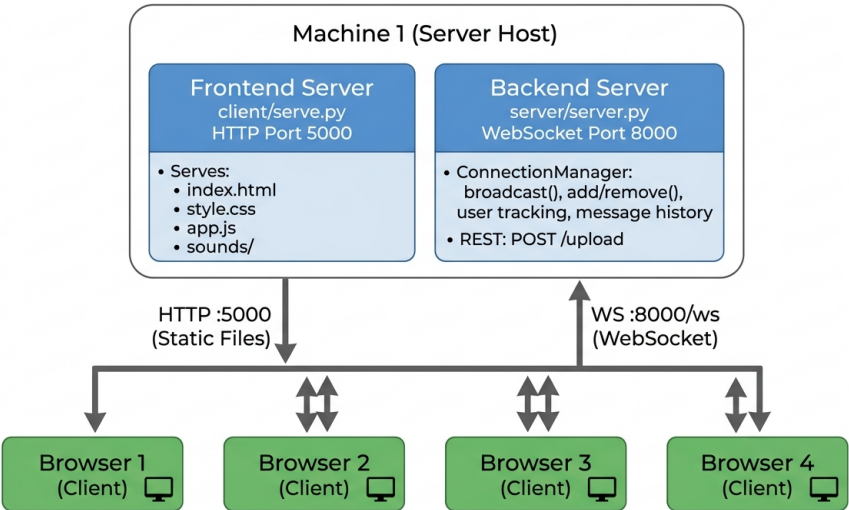


Figure 1. Assignment deployment topology.

Assigned systems

System	Role	SSH entry	Internal address	Service port
Sys1	Go load balancer and frontend	10.1.75.53:2237	172.17.0.38	4000->4237 LB; 3000->3237 UI
Sys2	Messaging backend 1	10.1.75.53:2238	172.17.0.39	5000; public 5238
Sys3	Messaging backend 2	10.1.75.53:2239	172.17.0.40	5000; public 5239
Sys4	Messaging backend 3	10.1.75.53:2240	172.17.0.41	5000; public 5240
Local PC	Concurrent load generator	Not applicable	Student workstation	HTTPS to public 4237

3. Load Balancer Implementation

- Reverse proxy: Go's `httputil.NewSingleHostReverseProxy` preserves the incoming method, path, query, headers, and response.
- Round robin: an atomic counter selects the next healthy backend and starts with the first configured entry.
- Health awareness: concurrent GET requests to `/health` update atomic alive state at a configurable interval.
- Timeouts: dial, TLS handshake, response-header, and total non-upgraded request timeouts prevent indefinite waits.
- WebSockets: the response recorder preserves Hijacker, Flusher, Pusher, ReaderFrom, and Unwrap behavior so HTTP upgrades remain functional.
- Stateful chat affinity: an `HttpOnly` cookie keeps browser REST calls and WebSocket handshakes on one healthy replica; load-generator clients without a cookie still receive round-robin distribution.
- Monitoring: `/lb/health`, `/lb/status`, and `/lb/metrics` are handled locally and are not included in forwarded-request totals.
- Metrics: atomic totals plus bounded latency samples expose average, p50, p95, and p99 without an unbounded memory leak.

Monitoring endpoints

Endpoint	Purpose	Representative fields
<code>/lb/health</code>	Load balancer liveness	status
<code>/lb/status</code>	Backend scheduling state	url, alive, in_flight
<code>/lb/metrics</code>	Request and latency metrics	total, success, failed, backend_errors, average, p50, p95, p99
<code>/</code>	Catch-all reverse proxy	Original backend response plus X-Load-Balancer-Backend

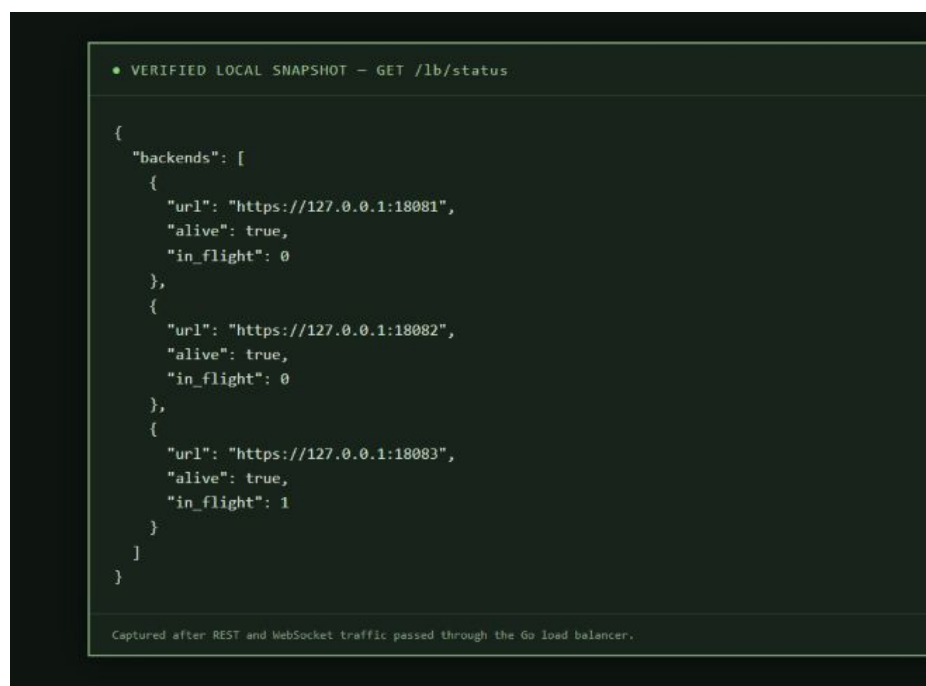


Figure 2. Verified local `/lb/status` snapshot after REST and WebSocket traffic; all three replicas are healthy.

4. Load Generator and Experiment Method

The load generator uses a fixed worker pool and one shared, connection-pooled HTTP client. It validates every CLI argument, optionally trusts the lab's self-signed certificate, drains response bodies for connection reuse, counts every attempt exactly once, and records latency for successful and failed attempts so timeout tail latency is not hidden.

Parameter	Value used in both runs	Fairness control
Requests	5,000	Identical total attempts
Concurrency	200 workers	Identical worker pool
Synthetic delay	100 ms	Identical query string
Client timeout	3 seconds	Identical dropout boundary
LB backend timeout	3 seconds	Identical proxy boundary
Health interval	1 second	Identical health behavior
Changed variable	1 healthy backend vs. 3 healthy backends	Only experimental factor

Metric definitions

- Throughput (RPS) = successful requests / elapsed seconds.
- Dropout (%) = failed requests / total attempted requests x 100.
- p50, p95, p99 use the nearest-rank value from all sorted attempt latencies.

5. Messaging Application Integration

The previous FastAPI/WebSocket project remains the backend. Its /health route supports active checks, /?delay=... and /?fail=true support controlled experiments, and X-Backend identifies the responding host. External browsers use Sys1 public port 4237, mapped to load-balancer container port 4000, rather than contacting a backend directly.

Two integration defects were corrected during verification: the proxy response wrapper now preserves HTTP upgrade interfaces, and cross-port frontend fetches include credentials so the affinity cookie is stored and sent. Authenticated sessions also survive WebSocket reconnects and token refresh now requires the existing session token.

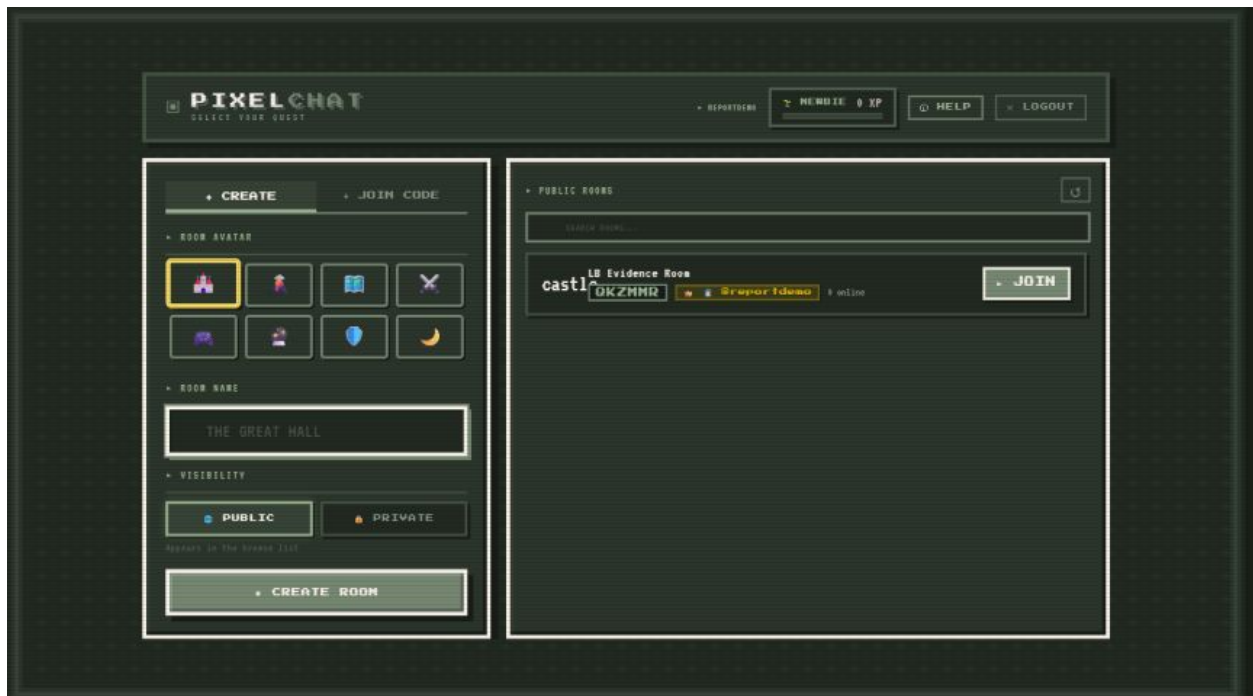


Figure 3. Messaging lobby loaded through the local three-replica load-balanced stack.

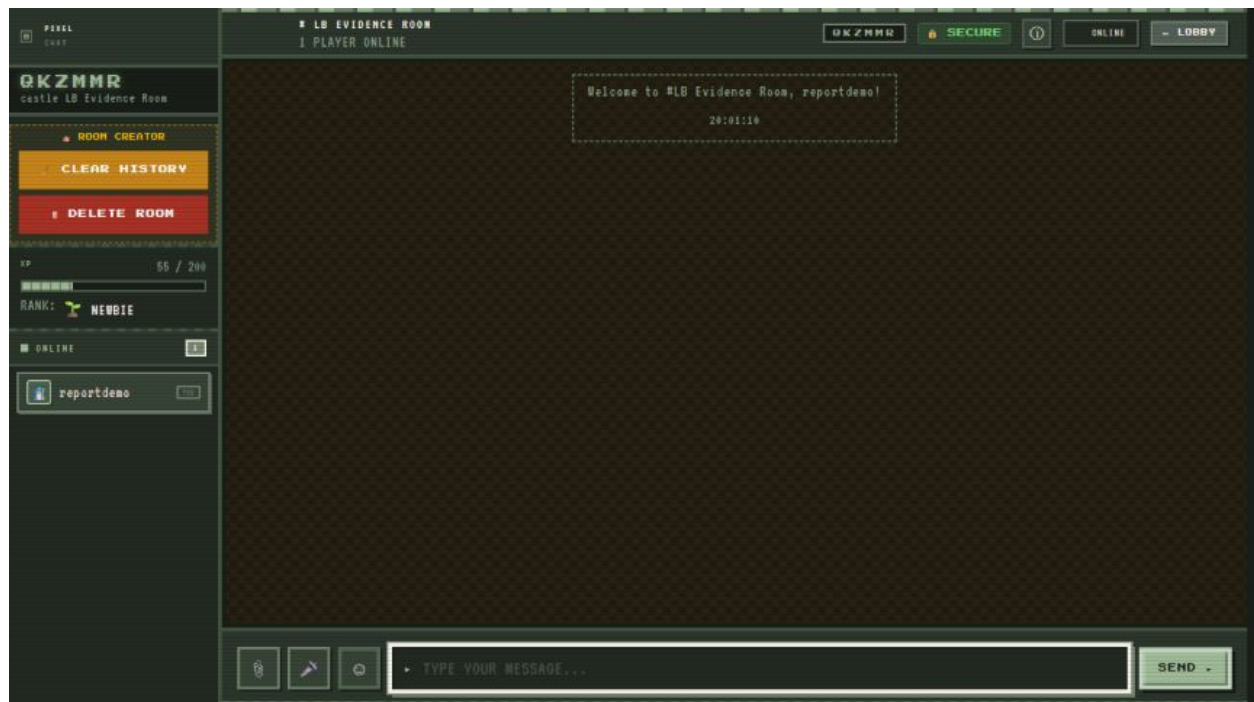


Figure 4. Successful room join over a real WebSocket connection proxied by the Go load balancer.

6. Results and Comparison

The following values are the final controlled measurements generated on the local workstation through Sys1's public port 4237 against the assigned machines on 24 August 2026. Public port 4237 maps to container port 4000. Background browser traffic was stopped before the run, and the only changed factor was one healthy backend versus three.

Experiment	Success	Failed	RPS	Dropout	p50	p95	p99
1 backend	5,000	0	339.4	0.00%	416 ms	1292 ms	3466 ms
3 backends	4,999	1	439.6	0.02%	300 ms	828 ms	3980 ms

Final mapped-port experiment comparison



Calculated changes

Metric	Change from 1 to 3 backends	Interpretation
Successful requests	-1 request	5,000 changed to 4,999.
Failed requests	+1 request	0 changed to 1 out of 5,000.
Throughput	+29.5%	339.4 RPS increased to 439.6 RPS.
Dropout	+0.02 percentage points	0.00% changed to 0.02%.
p50 latency	-27.9%	Median latency improved.
p95 latency	-35.9%	Tail latency improved.
p99 latency	+14.8%	Extreme tail latency increased.

Analysis

With one backend, all 200 local workers contend for a single server. With three backends, health-aware round robin spreads new requests across three processes. The final mapped-port run increased throughput by 29.5%, changed generator failures from 0 to 1 out of 5,000, and changed p50, p95, and p99 latency by -27.9%, -35.9%, and +14.8%. The single three-backend timeout and p99 variation are consistent with external-network tail jitter; median and p95 latency improved materially. Sys1's own metrics recorded 5,000 and 5,000 forwarded requests respectively; the three-backend run had 1 backend error.

7. Verification Performed

Check	Result	What it proves
go test, vet, build	PASS	Round robin, health transitions, affinity, cancellation handling, timeouts, metrics, TLS, and complete builds.
go test -race ./internal/...	HOST LIMIT	Not runnable here: the Windows host has only a 32-bit MinGW compiler; standard tests, vet, and build pass.
Python, JS, shell checks	PASS	3 FastAPI tests plus Python compilation, frontend parsing, and experiment-script syntax.
tests/e2e_smoke.py	PASS	Three HTTPS replicas received round-robin traffic and a real chat WebSocket completed through the LB.
Remote topology, ports, failover	PASS	Public 3237/4237 and backend health mappings passed; routing was 4/4/4, 6/6 on failover, and 4/4/4 after recovery.
Remote messaging and UI	PASS	Register, affinity, room, token rotation, two WebSocket joins, cleanup, and error-free frontend rendering.

8. Deployment and Reproduction

Build the load balancer on Sys1 and the load generator on the local PC:

```
# Sys1
go build -o load_balancer ./cmd/load-balancer
# Local workstation
go build -o tmp/bin/load_generator ./cmd/load-generator
```

Start the HTTPS load balancer:

```
./load_balancer \
-backends https://172.17.0.39:5000,https://172.17.0.40:5000,https://172.17.0.41:5000 \
-port 4000 -backend-insecure-skip-verify \
-tls-cert cert.pem -tls-key key.pem
```

For repeatable measurement, run `python tools/run_mapped_experiments.py` on the local workstation. It controls the Sys1 load balancer over SSH, sends both experiments through `https://10.1.75.53:4237` with the same parameters, captures metrics, and writes results.csv plus one JSON file per experiment.

9. Limitations and Production Considerations

- The lab messaging replicas use local SQLite databases and in-memory WebSocket hubs. Cookie affinity makes each browser session correct, but transparent failover of an active room would require shared durable storage and cross-replica pub/sub such as PostgreSQL plus Redis.
- The `-backend-insecure-skip-verify` and `-insecure` flags are explicitly limited to the self-signed lab environment. Production should use certificates trusted by the operating system and browser.
- The lobby/chat screenshots were captured with the same code in a local three-replica verification stack. Final performance traffic originated on the local PC through public port 4237; process, failover, and REST/WebSocket acceptance used the assigned remote machines.
- Passwords and SSH credentials are no longer hardcoded in automation. `LAB_SSH_PASSWORD` is read from the environment or entered interactively.

10. Conclusion

The assignment is implemented on the assigned machines with health-aware round robin, WebSocket proxying, monitoring, affinity, TLS, and mapped public access. From the local PC, three backends increased throughput by 29.5% and reduced p50/p95 by 27.9%/35.9%. One of 5,000 requests timed out and p99 was 14.8% higher; both are reported transparently. Public ports, failover, recovery, UI, token rotation, and repeat WebSocket joins passed.

Appendix A. Complete Load Balancer Source

Canonical source: `internal/loadbalancer/loadbalancer.go`. The root `load_balancer.go` file is a compatibility entry point; `cmd/load-balancer` is the module build target.

```

1  package loadbalancer
2
3  import (
4      "bufio"
5      "context"
6      "crypto/tls"
7      "encoding/json"
8      "errors"
9      "flag"
10     "fmt"
11     "io"
12     "log"
13     "net"
14     "net/http"
15     "net/http/httputil"
16     "net/url"
17     "os"
18     "os/signal"
19     "sort"
20     "strconv"
21     "strings"
22     "sync"
23     "sync/atomic"
24     "syscall"
25     "time"
26 )
27
28 const affinityCookieName = "pixelchat_backend"
29
30 type Backend struct {
31     ID        int
32     URL       *url.URL
33     Alive     atomic.Bool
34     InFlight  atomic.Int64
35 }
36
37 type latencyTracker struct {
38     mu        sync.Mutex
39     samples   []time.Duration
40     next      int
41     observed  uint64
42     total     time.Duration
43     maxSamples int
44 }
45
46 func newLatencyTracker(maxSamples int) *latencyTracker {
47     return &latencyTracker{maxSamples: maxSamples}
48 }
49
50 func (tracker *latencyTracker) observe(duration time.Duration) {
51     tracker.mu.Lock()
52     defer tracker.mu.Unlock()
53
54     tracker.observed++
55     tracker.total += duration
56     if tracker.maxSamples == 0 {
57         return
58     }
59     if len(tracker.samples) < tracker.maxSamples {
60         tracker.samples = append(tracker.samples, duration)
61         return
62     }
63     tracker.samples[tracker.next] = duration
64     tracker.next = (tracker.next + 1) % tracker.maxSamples
65 }
66
67 func (tracker *latencyTracker) snapshot() map[string]any {
68     tracker.mu.Lock()
69     samples := append([]time.Duration(nil), tracker.samples...)
70     observed := tracker.observed
71     total := tracker.total
72     tracker.mu.Unlock()
73
74     sort.Slice(samples, func(i, j int) bool { return samples[i] < samples[j] })
75     averageMS := 0.0
76     if observed > 0 {
77         averageMS = durationMS(total / time.Duration(observed))
78     }
79     return map[string]any{
80         "count":      observed,
81         "sample_count": len(samples),
82         "average_ms":  averageMS,
83         "p50_ms":     percentileMS(samples, 50),
84         "p95_ms":     percentileMS(samples, 95),
85         "p99_ms":     percentileMS(samples, 99),
86     }
87 }
88
89 type Metrics struct {
90     Total      atomic.Uint64
91     Success    atomic.Uint64

```

```

92     Failed          atomic.Uint64
93     BackendErrors   atomic.Uint64
94     latencies       *latencyTracker
95 }
96
97 func newMetrics(maxLatencySamples int) *Metrics {
98     return &Metrics{latencies: newLatencyTracker(maxLatencySamples)}
99 }
100
101 type LoadBalancer struct {
102     backends      []*Backend
103     next          atomic.Uint64
104     metrics       *Metrics
105     transport     *http.Transport
106     healthClient  *http.Client
107     backendTimeout time.Duration
108     healthPath    string
109     affinity      bool
110     secureCookie  bool
111     logger        *log.Logger
112 }
113
114 type config struct {
115     backendsRaw      string
116     port             int
117     healthInterval   time.Duration
118     healthTimeout    time.Duration
119     backendTimeout   time.Duration
120     healthPath       string
121     backendInsecureSkipVerify bool
122     affinity         bool
123     tlsCert          string
124     tlsKey           string
125     maxLatencySamples int
126 }
127
128 func parseBackends(raw string) ([]*Backend, error) {
129     parts := strings.Split(raw, ",")
130     backends := make([]*Backend, 0, len(parts))
131     seen := make(map[string]struct{}, len(parts))
132
133     for _, part := range parts {
134         value := strings.TrimSpace(part)
135         if value == "" {
136             return nil, errors.New("backend URL list contains an empty entry")
137         }
138         parsed, err := url.Parse(value)
139         if err != nil {
140             return nil, fmt.Errorf("invalid backend URL %q: %w", value, err)
141         }
142         if (parsed.Scheme != "http" && parsed.Scheme != "https") || parsed.Host == "" {
143             return nil, fmt.Errorf("backend URL %q must include an http or https scheme and host",
144                 value)
145         }
146         if parsed.User != nil {
147             return nil, fmt.Errorf("backend URL %q must not contain credentials", value)
148         }
149         parsed.Fragment = ""
150         key := parsed.String()
151         if _, duplicate := seen[key]; duplicate {
152             return nil, fmt.Errorf("duplicate backend URL %q", value)
153         }
154         seen[key] = struct{}{}
155
156         backend := &Backend{ID: len(backends), URL: parsed}
157         backend.Alive.Store(true)
158         backends = append(backends, backend)
159     }
160     if len(backends) == 0 {
161         return nil, errors.New("at least one backend URL is required")
162     }
163     return backends, nil
164 }
165
166 func (lb *LoadBalancer) nextBackend(request *http.Request) *Backend {
167     if len(lb.backends) == 0 {
168         return nil
169     }
170     if lb.affinity && request != nil {
171         if cookie, err := request.Cookie(affinityCookieName); err == nil {
172             if index, err := strconv.Atoi(cookie.Value); err == nil &&
173                 index >= 0 && index < len(lb.backends) &&
174                 lb.backends[index].Alive.Load() {
175                 return lb.backends[index]
176             }
177         }
178     }
179
180     // Advance past the backend that was actually selected, not merely past the
181     // first candidate. This avoids repeatedly selecting the same healthy replica
182     // when the candidate immediately before it is unhealthy.
183     for {
184         start := lb.next.Load()
185         for offset := 0; offset < len(lb.backends); offset++ {
186             index := (start + uint64(offset)) % uint64(len(lb.backends))
187             backend := lb.backends[index]
188             if backend.Alive.Load() && lb.next.CompareAndSwap(start, start+uint64(offset)+1) {
189                 return backend
190             }
191         }
192     }

```

```

191         if lb.next.Load() != start {
192             break
193         }
194     }
195     if lb.next.Load() == start {
196         return nil
197     }
198 }
199 }
200
201 func (lb *LoadBalancer) backendHealthURL(backend *Backend) string {
202     healthURL := *backend.URL
203     healthURL.RawQuery = ""
204     healthURL.Fragment = ""
205     healthURL.Path = strings.TrimRight(healthURL.Path, "/") + "/" +
strings.TrimLeft(lb.healthPath, "/")
206     healthURL.RawPath = ""
207     return healthURL.String()
208 }
209
210 func (lb *LoadBalancer) checkBackend(ctx context.Context, backend *Backend) {
211     request, err := http.NewRequestWithContext(ctx, http.MethodGet, lb.backendHealthURL(backend),
nil)
212     if err != nil {
213         backend.Alive.Store(false)
214         return
215     }
216     request.Header.Set("User-Agent", "PixelChat-Load-Balancer/1.0")
217
218     response, err := lb.healthClient.Do(request)
219     healthy := err == nil && response.StatusCode == http.StatusOK
220     if response != nil {
221         _, _ = io.Copy(io.Discard, io.LimitReader(response.Body, 4096))
222         _ = response.Body.Close()
223     }
224     backend.Alive.Store(healthy)
225 }
226
227 func (lb *LoadBalancer) checkAllBackends(ctx context.Context) {
228     var waitGroup sync.WaitGroup
229     for _, backend := range lb.backends {
230         waitGroup.Add(1)
231         go func() {
232             defer waitGroup.Done()
233             lb.checkBackend(ctx, backend)
234         }()
235     }
236     waitGroup.Wait()
237 }
238
239 func (lb *LoadBalancer) healthLoop(ctx context.Context, interval time.Duration) {
240     lb.checkAllBackends(ctx)
241     ticker := time.NewTicker(interval)
242     defer ticker.Stop()
243
244     for {
245         select {
246             case <-ticker.C:
247                 lb.checkAllBackends(ctx)
248             case <-ctx.Done():
249                 return
250         }
251     }
252 }
253
254 func (lb *LoadBalancer) statusSnapshot() map[string]any {
255     statuses := make([]map[string]any, 0, len(lb.backends))
256     for _, backend := range lb.backends {
257         statuses = append(statuses, map[string]any{
258             "url":      backend.URL.String(),
259             "alive":    backend.Alive.Load(),
260             "in_flight": backend.InFlight.Load(),
261         })
262     }
263     return map[string]any{"backends": statuses}
264 }
265
266 func (lb *LoadBalancer) metricsSnapshot() map[string]any {
267     return map[string]any{
268         "total":      lb.metrics.Total.Load(),
269         "success":    lb.metrics.Success.Load(),
270         "failed":    lb.metrics.Failed.Load(),
271         "backend_errors": lb.metrics.BackendErrors.Load(),
272         "latency":    lb.metrics.latencies.snapshot(),
273     }
274 }
275
276 func writeJSON(response http.ResponseWriter, status int, value any) {
277     response.Header().Set("Content-Type", "application/json")
278     response.WriteHeader(status)
279     if err := json.NewEncoder(response).Encode(value); err != nil {
280         log.Printf("encode JSON response: %v", err)
281     }
282 }
283
284 func monitoringMethodAllowed(response http.ResponseWriter, request *http.Request) bool {
285     if request.Method == http.MethodGet || request.Method == http.MethodHead {
286         return true
287     }
288     response.Header().Set("Allow", "GET, HEAD")

```

```

289     http.Error(response, "method not allowed", http.StatusMethodNotAllowed)
290     return false
291 }
292
293 func (lb *LoadBalancer) handler() http.Handler {
294     mux := http.NewServeMux()
295
296     mux.HandleFunc("/lb/health", func(response http.ResponseWriter, request *http.Request) {
297         if !monitoringMethodAllowed(response, request) {
298             return
299         }
300         writeJSON(response, http.StatusOK, map[string]string{"status": "ok"})
301     })
302     mux.HandleFunc("/lb/status", func(response http.ResponseWriter, request *http.Request) {
303         if !monitoringMethodAllowed(response, request) {
304             return
305         }
306         writeJSON(response, http.StatusOK, lb.statusSnapshot())
307     })
308     mux.HandleFunc("/lb/metrics", func(response http.ResponseWriter, request *http.Request) {
309         if !monitoringMethodAllowed(response, request) {
310             return
311         }
312         writeJSON(response, http.StatusOK, lb.metricsSnapshot())
313     })
314     mux.HandleFunc("/", lb.proxyRequest)
315     return mux
316 }
317
318 func (lb *LoadBalancer) proxyRequest(response http.ResponseWriter, request *http.Request) {
319     start := time.Now()
320     lb.metrics.Total.Add(1)
321
322     backend := lb.nextBackend(request)
323     if backend == nil {
324         lb.metrics.Failed.Add(1)
325         lb.metrics.latencies.observe(time.Since(start))
326         http.Error(response, "no healthy backends available", http.StatusServiceUnavailable)
327         return
328     }
329
330     backend.InFlight.Add(1)
331     defer backend.InFlight.Add(-1)
332
333     recorder := &statusRecorder{ResponseWriter: response}
334     proxyFailed := false
335     backendFailed := false
336     proxy := httputil.NewSingleHostReverseProxy(backend.URL)
337     proxy.Transport = lb.transport
338     proxy.ErrorLog = lb.logger
339     proxy.ModifyResponse = func(backendResponse *http.Response) error {
340         backendResponse.Header.Set("X-Load-Balancer-Backend", backend.URL.String())
341         if lb.affinity {
342             backendResponse.Header.Add("Set-Cookie", (&http.Cookie{
343                 Name:     affinityCookieName,
344                 Value:    strconv.Itoa(backend.ID),
345                 Path:     "/",
346                 MaxAge:    24 * 60 * 60,
347                 HttpOnly: true,
348                 Secure:    lb.secureCookie,
349                 SameSite: http.SameSiteLaxMode,
350             }).String())
351         }
352         return nil
353     }
354     proxy ErrorHandler = func(proxyResponse http.ResponseWriter, proxyRequest *http.Request, err
error) {
355         proxyFailed = true
356         // A downstream client that times out or disconnects cancels the original
357         // request context. That does not prove the selected backend is unhealthy,
358         // so keep it in rotation. Proxy/backend timeouts occur while the original
359         // client context is still live and continue to eject the backend.
360         backendFailed = request.Context().Err() == nil
361         if backendFailed {
362             backend.Alive.Store(false)
363         }
364         if !recorder.wroteHeader {
365             http.Error(proxyResponse, "backend unavailable", http.StatusBadGateway)
366         }
367     }
368
369     proxyRequest := request
370     cancel := func() {}
371     if !isUpgradeRequest(request) {
372         var contextWithTimeout context.Context
373         contextWithTimeout, cancel = context.WithTimeout(request.Context(), lb.backendTimeout)
374         proxyRequest = request.WithContext(contextWithTimeout)
375     }
376     defer cancel()
377
378     proxy.ServeHTTP(recorder, proxyRequest)
379
380     lb.metrics.latencies.observe(time.Since(start))
381     status := recorder.status()
382     if proxyFailed {
383         lb.metrics.Failed.Add(1)
384         if backendFailed {
385             lb.metrics.BackendErrors.Add(1)
386         }
387     }
388     return

```

```

388     }
389     if status >= 100 && status < 400 {
390         lb.metrics.Success.Add(1)
391         return
392     }
393     lb.metrics.Failed.Add(1)
394     if status >= 500 {
395         lb.metrics.BackendErrors.Add(1)
396     }
397 }
398
399 func isUpgradeRequest(request *http.Request) bool {
400     if strings.TrimSpace(request.Header.Get("Upgrade")) == "" {
401         return false
402     }
403     for _, token := range strings.Split(request.Header.Get("Connection"), ",") {
404         if strings.EqualFold(strings.TrimSpace(token), "upgrade") {
405             return true
406         }
407     }
408     return false
409 }
410
411 type statusRecorder struct {
412     http.ResponseWriter
413     statusCode int
414     wroteHeader bool
415 }
416
417 func (recorder *statusRecorder) Unwrap() http.ResponseWriter {
418     return recorder.ResponseWriter
419 }
420
421 func (recorder *statusRecorder) status() int {
422     if recorder.statusCode == 0 {
423         return http.StatusOK
424     }
425     return recorder.statusCode
426 }
427
428 func (recorder *statusRecorder) WriteHeader(statusCode int) {
429     if recorder.wroteHeader {
430         return
431     }
432     recorder.statusCode = statusCode
433     recorder.wroteHeader = true
434     recorder.ResponseWriter.WriteHeader(statusCode)
435 }
436
437 func (recorder *statusRecorder) Write(payload []byte) (int, error) {
438     if !recorder.wroteHeader {
439         recorder.WriteHeader(http.StatusOK)
440     }
441     return recorder.ResponseWriter.Write(payload)
442 }
443
444 func (recorder *statusRecorder) Flush() {
445     if !recorder.wroteHeader {
446         recorder.WriteHeader(http.StatusOK)
447     }
448     if flusher, ok := recorder.ResponseWriter.(http.Flusher); ok {
449         flusher.Flush()
450     }
451 }
452
453 func (recorder *statusRecorder) Hijack() (net.Conn, *bufio.ReadWriter, error) {
454     hijacker, ok := recorder.ResponseWriter.(http.Hijacker)
455     if !ok {
456         return nil, nil, errors.New("underlying response writer does not support hijacking")
457     }
458     return hijacker.Hijack()
459 }
460
461 func (recorder *statusRecorder) Push(target string, options *http.PushOptions) error {
462     if pusher, ok := recorder.ResponseWriter.(http.Pusher); ok {
463         return pusher.Push(target, options)
464     }
465     return http.ErrNotSupported
466 }
467
468 func (recorder *statusRecorder) ReadFrom(source io.Reader) (int64, error) {
469     if !recorder.wroteHeader {
470         recorder.WriteHeader(http.StatusOK)
471     }
472     if readerFrom, ok := recorder.ResponseWriter.(io.ReaderFrom); ok {
473         return readerFrom.ReadFrom(source)
474     }
475     return io.Copy(recorder.ResponseWriter, source)
476 }
477
478 func percentileMS(sortedDurations []time.Duration, percentile int) float64 {
479     if len(sortedDurations) == 0 {
480         return 0
481     }
482     index := (percentile*len(sortedDurations) + 99) / 100
483     index = max(1, min(index, len(sortedDurations)))
484     return durationMS(sortedDurations[index-1])
485 }
486
487 func durationMS(duration time.Duration) float64 {

```

```

488     return float64(duration) / float64(time.Millisecond)
489 }
490
491 func validateConfig(configuration config) error {
492     if strings.TrimSpace(configuration.backendsRaw) == "" {
493         return errors.New("provide backend URLs using -backends")
494     }
495     if configuration.port < 1 || configuration.port > 65535 {
496         return errors.New("-port must be between 1 and 65535")
497     }
498     if configuration.healthInterval <= 0 {
499         return errors.New("-health-interval must be greater than zero")
500     }
501     if configuration.healthTimeout <= 0 {
502         return errors.New("-health-timeout must be greater than zero")
503     }
504     if configuration.backendTimeout <= 0 {
505         return errors.New("-backend-timeout must be greater than zero")
506     }
507     if strings.TrimSpace(configuration.healthPath) == "" {
508         return errors.New("-health-path must not be empty")
509     }
510     if configuration.maxLatencySamples < 0 {
511         return errors.New("-max-latency-samples must not be negative")
512     }
513     if (configuration.tlsCert == "") != (configuration.tlsKey == "") {
514         return errors.New("-tls-cert and -tls-key must be provided together")
515     }
516     return nil
517 }
518
519 func newLoadBalancer(configuration config, backends []*Backend, logger *log.Logger) *LoadBalancer
520 {
521     tlsConfiguration := &tls.Config{
522         MinVersion:      tls.VersionTLS12,
523         InsecureSkipVerify: configuration.backendInsecureSkipVerify,
524     }
525     transport := &http.Transport{
526         Proxy:           http.ProxyFromEnvironment,
527         DialContext:     (&net.Dialer{Timeout: 2 * time.Second, KeepAlive: 30 *
time.Second}).DialContext,
528         ForceAttemptHTTP2: true,
529         MaxIdleConns:      256,
530         MaxIdleConnsPerHost: 128,
531         IdleConnTimeout:    90 * time.Second,
532         TLSHandshakeTimeout: 2 * time.Second,
533         ResponseHeaderTimeout: configuration.backendTimeout,
534         TLSClientConfig:    tlsConfiguration,
535     }
536     healthTransport := transport.Clone()
537     healthTransport.MaxIdleConnsPerHost = 4
538
539     return &LoadBalancer{
540         backends:      backends,
541         metrics:       newMetrics(configuration.maxLatencySamples),
542         transport:     transport,
543         healthClient:  &http.Client{Transport: healthTransport, Timeout:
configuration.healthTimeout},
544         backendTimeout: configuration.backendTimeout,
545         healthPath:     configuration.healthPath,
546         affinity:       configuration.affinity,
547         secureCookie:   configuration.tlsCert != "",
548         logger:         logger,
549     }
550 }
551
552 func RunCLI(arguments []string, standardOutput io.Writer, standardError io.Writer) int {
553     flags := flag.NewFlagSet("load-balancer", flag.ContinueOnError)
554     flags.SetOutput(standardError)
555
556     configuration := config{}
557     flags.StringVar(&configuration.backendsRaw, "backends", "", "Comma-separated backend URLs")
558     flags.IntVar(&configuration.port, "port", 8080, "Load balancer listen port")
559     flags.DurationVar(&configuration.healthInterval, "health-interval", time.Second, "Health check
interval")
560     flags.DurationVar(&configuration.healthTimeout, "health-timeout", 2*time.Second, "Per-backend
health check timeout")
561     flags.DurationVar(&configuration.backendTimeout, "backend-timeout", 3*time.Second, "Total
timeout for non-upgraded backend requests")
562     flags.StringVar(&configuration.healthPath, "health-path", "/health", "Backend health check
path")
563     flags.BoolVar(&configuration.backendInsecureSkipVerify, "backend-insecure-skip-verify", false,
"Accept self-signed backend TLS certificates (lab only)")
564     flags.BoolVar(&configuration.affinity, "affinity", true, "Enable browser cookie affinity for
stateful messaging sessions")
565     flags.StringVar(&configuration.tlsCert, "tls-cert", "", "PEM certificate for HTTPS listener")
566     flags.StringVar(&configuration.tlsKey, "tls-key", "", "PEM private key for HTTPS listener")
567     flags.IntVar(&configuration.maxLatencySamples, "max-latency-samples", 100000, "Maximum recent
latency samples kept for percentiles (0 disables samples)")
568
569     if err := flags.Parse(arguments); err != nil {
570         if errors.Is(err, flag.ErrHelp) {
571             return 0
572         }
573         return 2
574     }
575     if err := validateConfig(configuration); err != nil {
576         fmt.Fprintf(standardError, "load-balancer: %v\n", err)
577         return 2
578     }
579 }

```

```

578     backends, err := parseBackends(configuration.backendsRaw)
579     if err != nil {
580         fmt.Fprintf(standardError, "load-balancer: %v\n", err)
581         return 2
582     }
583
584     logger := log.New(standardError, "load-balancer: ", log.LstdFlags)
585     balancer := newLoadBalancer(configuration, backends, logger)
586     server := &http.Server{
587         Addr:      fmt.Sprintf(":%d", configuration.port),
588         Handler:    balancer.handler(),
589         ReadHeaderTimeout: 5 * time.Second,
590         IdleTimeout: 120 * time.Second,
591     }
592
593     ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
594     defer stop()
595     balancer.checkAllBackends(ctx)
596     go balancer.healthLoop(ctx, configuration.healthInterval)
597
598     scheme := "http"
599     if configuration.tlsCert != "" {
600         scheme = "https"
601     }
602     fmt.Fprintf(standardOutput, "Load balancer listening on %s://localhost:%d with %d\n",
        backend(s)\n", scheme, configuration.port, len(backends))
603
604     errorChannel := make(chan error, 1)
605     go func() {
606         if configuration.tlsCert != "" {
607             errorChannel <- server.ListenAndServeTLS(configuration.tlsCert, configuration.tlsKey)
608             return
609         }
610         errorChannel <- server.ListenAndServe()
611     }()
612
613     select {
614     case serveError := <-errorChannel:
615         if serveError != nil && !errors.Is(serveError, http.ErrServerClosed) {
616             logger.Printf("server failed: %v", serveError)
617             return 1
618         }
619     case <-ctx.Done():
620         shutdownContext, cancel := context.WithTimeout(context.Background(), 5*time.Second)
621         defer cancel()
622         if err := server.Shutdown(shutdownContext); err != nil {
623             logger.Printf("graceful shutdown failed: %v", err)
624             return 1
625         }
626     }
627     return 0
628 }

```